

Tema 1: Fundamentos de Análisis de Datos y Técnicas Básicas

18/09/2026



Big Data



5 V's

Doug Laney, analista de Gartner, introduzco en un artículo del 2001 estas cinco características como marco básico para entender como funciona el Big Data-

Desde entonces se consideran estas 5 características las claves que definen este campo de la informática.



Volumen
Variedad
Velocidad
Veracidad
Valor



Aplicaciones Big Data



Salud

Finanzas

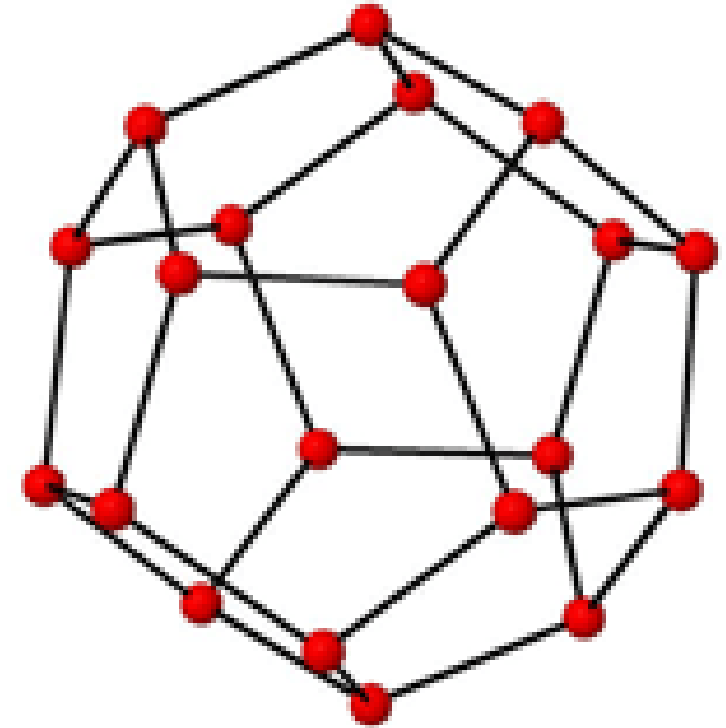
Comercio online

Rutas, transporte

IoT



Matemática discreta



Matemática discreta

1. • Unión (U): unión de dos conjuntos A y B es el conjunto que contiene elementos que están en A, B o ambos. En análisis de datos puede representar combinación de bases de datos distintas.
2. • Intersección ($\cap$): intersección de dos conjuntos A y B es el conjunto que contiene todos los elementos que están en A y en B. En análisis de datos sirve para detectar elementos comunes en dos datasets.
3. • Diferencia (-): la intersección de dos conjuntos A y B es el conjunto que contiene elementos que están en A y que no están en B. Podría utilizarse para encontrar información de clientes que han hecho unas compras en un supermercado y en otro no.

Conjuntos



Matemática discreta

1. Relaciones binarias: Una relación binaria entre dos conjuntos A y B es un subconjunto del producto cartesiano $A \times B$.
2. Propiedades de relaciones:
 - Reflexiva: Una relación es reflexiva si cada elemento está relacionado consigo mismo.
 - Simétrica: Una relación es simétrica si, siempre que un elemento A esté relacionado con un elemento B, B también está relacionado con A.
 - Transitiva: Una relación es transitiva si, cuando un elemento A está relacionado con B y B está relacionado con C, A también está relacionado con C.

Relaciones

R es reflexiva $\Leftrightarrow \forall x \in A; x \sim x$

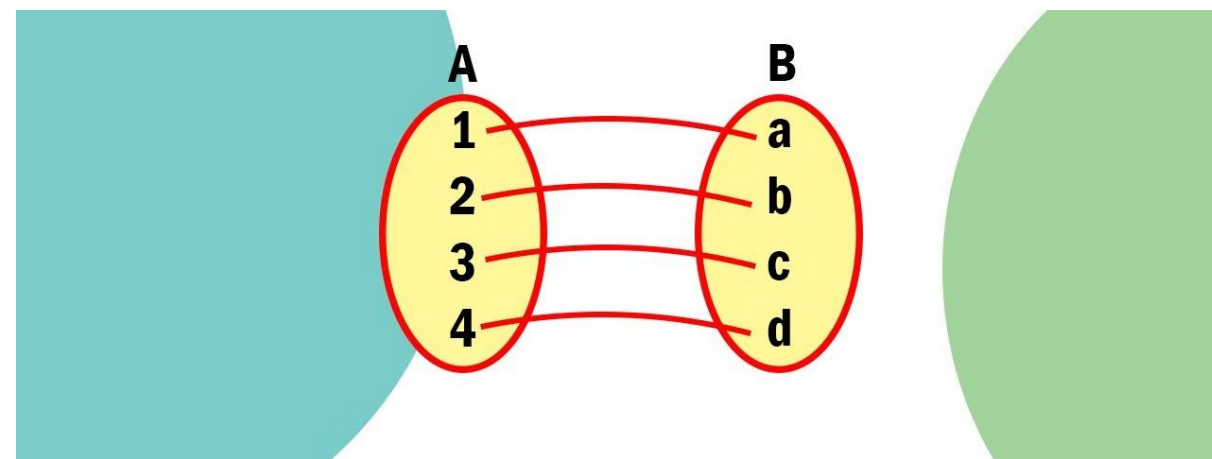
R es simétrica $\Leftrightarrow \forall x, y \in A; x \sim y \Rightarrow y \sim x$

R es transitiva $\Leftrightarrow \forall x, y, z \in A; x \sim y \wedge y \sim z \Rightarrow x \sim z$

Matemática discreta

1. Funciones: relación especial que asigna a cada elemento de un conjunto A un único elemento del conjunto B.

Funciones



Matemática discreta

1. Permutaciones: diferentes maneras en las que se pueden ordenar o disponer los elementos de un conjunto específico, donde el orden importa. Es decir, si tienes un conjunto de elementos, cada permutación es una forma diferente de organizar esos elementos.
2. Combinaciones: diferentes maneras en que se pueden seleccionar elementos de un conjunto, sin importar el orden. Aquí, el enfoque no está en cómo se ordenan los elementos, sino simplemente en qué elementos se seleccionan.

La combinatoria es útil en el campo del Big Data porque permite seleccionar muestras representativas de grandes volúmenes de datos y para identificar patrones a partir de ciertas combinaciones.

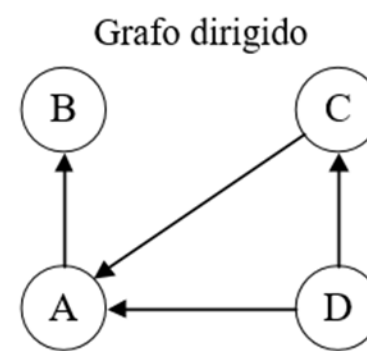
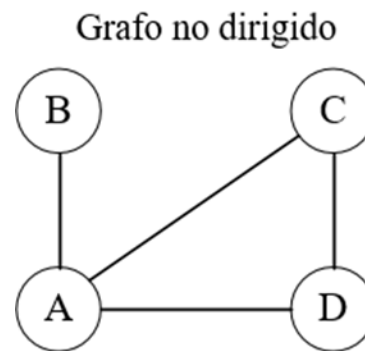
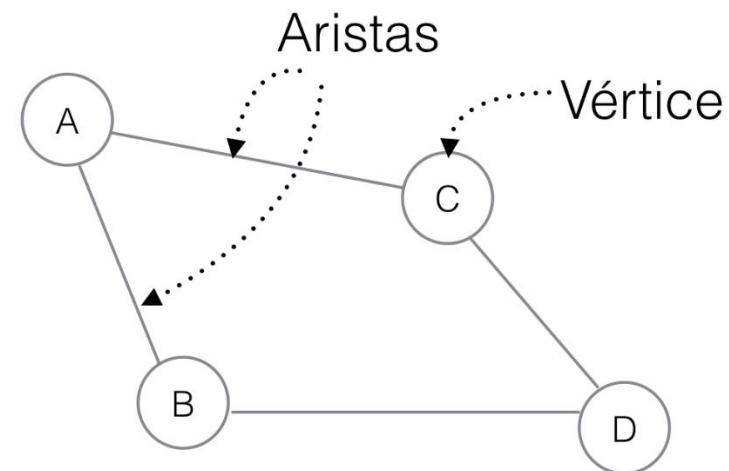
Combinatoria

Permutación	Resultado	
A $\swarrow$ B $\searrow$ C	C	ABC
	B	ACB
B $\swarrow$ A $\searrow$ C	C	BAC
	A	BCA
C $\swarrow$ A $\searrow$ B	B	CAB
	A	CBA

Matemática discreta

Un grafo es una estructura que consiste en un conjunto de nodos (o vértices) y un conjunto de aristas (o enlaces) que conectan pares de nodos. Los grafos son fundamentales para modelar relaciones y redes en datos.

Grafos



Matemática discreta

El álgebra booleana es la rama de las matemáticas discretas que se centra en estudiar las variables que toman valores verdaderos o falsos (0 o 1).

Las operaciones básicas son el AND ($\wedge$), OR ($\vee$), NOT ($\neg$), e implicación ($\rightarrow$).

Esta álgebra booleana tiene varias utilidades, se usa en la construcción de consultas SQL para extraer datos y pueden servir para filtrar datos de condiciones lógicas.

Álgebra booleana

Compuerta	Símbolo	Tabla de verdad	Expresión																		
AND		<table><tr><th colspan="2">Entrada</th><th>Salida</th></tr><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Entrada		Salida	A	B	X	0	0	0	0	1	0	1	0	0	1	1	1	$X = AB$
Entrada		Salida																			
A	B	X																			
0	0	0																			
0	1	0																			
1	0	0																			
1	1	1																			
OR		<table><tr><th colspan="2">Entrada</th><th>Salida</th></tr><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Entrada		Salida	A	B	X	0	0	0	0	1	1	1	0	1	1	1	1	$X = A + B$
Entrada		Salida																			
A	B	X																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	1																			
NOT		<table><tr><th>Entrada</th><th>Salida</th></tr><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	Entrada	Salida	A	X	0	1	1	0	$X = \bar{A}$										
Entrada	Salida																				
A	X																				
0	1																				
1	0																				

Lógica algorítmica

- AND ($\wedge$): Ambas proposiciones deben ser verdaderas.
- OR ($\vee$): Al menos una de las proposiciones debe ser verdadera.
- NOT ($\neg$): Invierte el valor de verdad de la proposición.
- Implicación ($\rightarrow$): Si la primera proposición es verdadera, la segunda también debe serlo.

Lógica proposicional

Lógica algorítmica

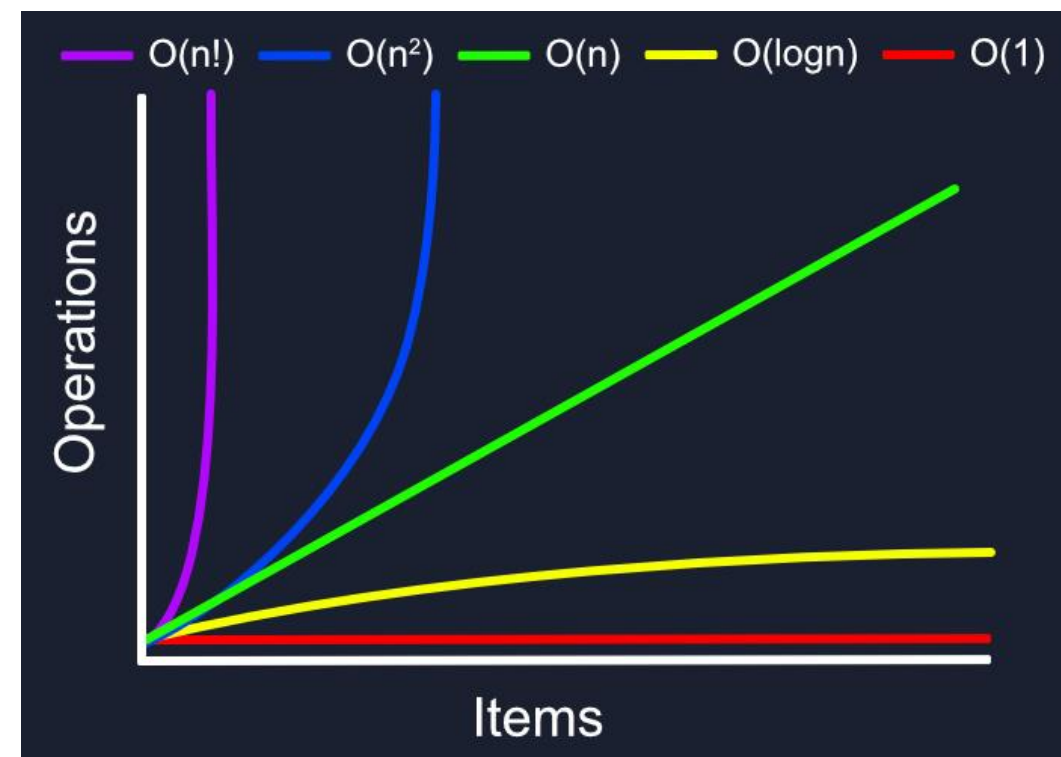
- Cuantificador Universal ($\forall$): Expresa que una proposición es verdadera para todos los elementos de un dominio. Ejemplo: "Todos los clientes realizaron una compra en el último mes".
- Cuantificador Existencial ($\exists$): Indica que existe al menos un elemento en el dominio para el cual la proposición es verdadera. Ejemplo: "Existe al menos un cliente que realizó una compra de más de 100 euros".

Lógica de predicados

Lógica algorítmica

- $O(1)$: Tiempo constante, no importa el tamaño de entrada.
- $O(n)$: Tiempo lineal, donde el tiempo de ejecución crece linealmente con el tamaño de entrada.
- $O(n^2)$: Tiempo cuadrático, donde el tiempo de ejecución crece de manera cuadrática con el tamaño de la entrada.
- $O(2^n)$: Tiempo Exponencial, el tiempo de ejecución se duplica con cada incremento en el tamaño de la entrada.
- $O(\log n)$: Tiempo Logarítmico, el tiempo de ejecución crece logarítmicamente con el tamaño de la entrada.

Complejidad Algorítmica



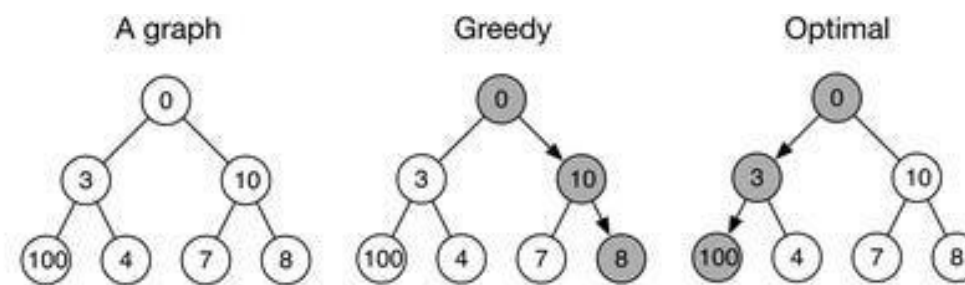
Lógica algorítmica

Métodos de resolución de problemas que, en vez de buscar soluciones óptimas, buscan soluciones suficientemente buenas en un tiempo razonable.

Por ejemplo, el “greedy” es un algoritmo que en cada paso elige la opción mejor, pero sin tener en cuenta que pasará más adelante (puede resultar no ser la mejor).

crece logarítmicamente con el tamaño de la entrada.

Heurísticas



A greedy algorithm fails to maximise the sum of nodes along a path from the top to the bottom because it lacks the foresight to choose suboptimal solutions in the current iteration that will allow for better solutions later

ETL

Extract, transform, load

- **Extract (extracción):** fase que consiste en recoger datos de varias fuentes. El objetivo es asegurar que los datos más relevantes sean reunidos para su posterior análisis.
- **Transform (transformación):** fase en que se limpian los datos previamente extraídos y se convierten al formato adecuado. Algunos aspectos que se pueden trabajar son la eliminación de duplicados, el manejo de datos faltantes, la conversión de formatos de datos o la normalización.
- **Carga (load):** los datos, una vez extraídos y transformados, son cargados en el sitio de almacenamiento destino. Ahora los datos ya están disponibles para realizar el análisis o para ser usados en un modelo de IA.



Extracción Datos estructurados

- Consultas SQL
- APIs de bases de datos

```
1 SELECT * FROM ventas
2 WHERE fecha_venta >= '2023-07-01' AND fecha_venta <= '2023-07-31';
```

```
import pandas as pd
from sqlalchemy import create_engine

# Conexión a la base de datos
engine = create_engine('postgresql://usuario:contraseña@localhost:5432/mi_base_datos')
df = pd.read_sql_query("SELECT * FROM ventas WHERE fecha_venta >= '2023-07-01'", engine)
```

Extracción

Datos semiestructurados

- Lectura archivos JSON
- Scraping web

```
1 import json
2
3 with open('datos.json', 'r') as file:
4     data = json.load(file)
5
6 print(data)
7
```

Extracción Datos no estructurados

- Procesamiento de texto
- Análisis de imágenes

```
3 import spacy
4
5 # Cargamos el modelo en español
6 nlp = spacy.load("es_core_news_sm")
7
8 texto = "Antonio no trajo las rosas a María."
9 doc = nlp(texto)
10
11 for entidad in doc.ents:
12     print(entidad.text, entidad.label_)
13
```

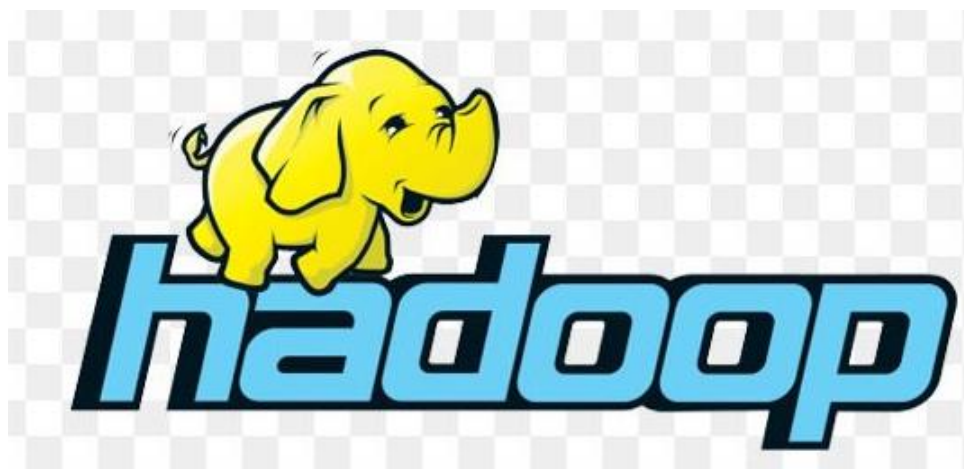
Extracción

Datos a tiempo real

- Streams de datos
- APIs de streaming

```
1 import tweepy
2
3 # Autenticación
4 auth = tweepy.OAuth1UserHandler(consumer_key, consumer_secret, access_token, access_token_secret)
5 api = tweepy.API(auth)
6
7 # Definimos un listener de streaming
8 class MyStreamListener(tweepy.StreamListener):
9     def on_status(self, status):
10         print(status.text)
11
12 # Iniciamos el stream
13 myStreamListener = MyStreamListener()
14 myStream = tweepy.Stream(auth=api.auth, listener=myStreamListener)
15 myStream.filter(track=['bigdata'])
16
```

Herramientas



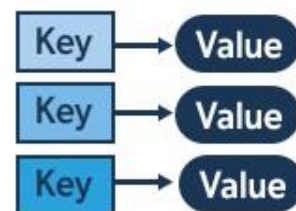
Hadoop - Spark



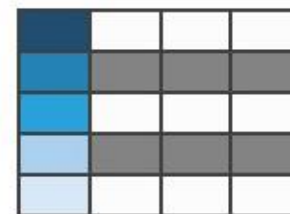
NoSQL

NoSQL

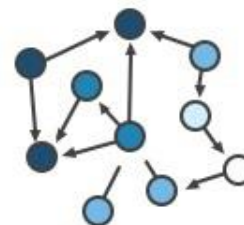
Key-Value



Column-Family



Graph



Document



Preprocesamiento

Limpieza de datos

Transformación de los datos

Integración de datos

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'ID': [1, 2, 2, 3],
5     'Nombre': ['Ana', 'Luis', 'Luis', 'Eva']
6 })
7
8 df_cleaned = df.drop_duplicates()
9 print(df_cleaned)
```

```
1 import pandas as pd
2
3 # DataFrame de ejemplo
4 df = pd.DataFrame({'A': [1, 2, None], 'B': [4, None, 6]})
5
6 # Eliminar filas con valores nulos
7 df_sin_nulos = df.dropna()
8
9 # Mostrar el DataFrame sin valores nulos
10 print(df_sin_nulos)
```

```
1 from sklearn.preprocessing import MinMaxScaler
2
3 scaler = MinMaxScaler()
4 df_scaled = scaler.fit_transform(df[['Edad', 'Ingreso']])
5
```

Lenguajes Marcaje

HTML -> crear y estructurar contenido en la web.

XML -> comunicación entre sistemas.

YAML -> lenguaje usado en IA.

```
<!DOCTYPE html>
<html>
<!-- created 2010-01-01 -->
  <head>
    <title>sample</title>
  </head>
  <body>
    <p>Voluptatem accusantium
      totam rem aperiam.</p>
  </body>
</html>
```

HTML

Lenguajes Marcaje

HTML -> crear y estructurar contenido en la web.

XML -> comunicación entre sistemas.

YAML -> lenguaje usado en IA.

```
<!DOCTYPE html>
<html>
<!-- created 2010-01-01 -->
  <head>
    <title>sample</title>
  </head>
  <body>
    <p>Voluptatem accusantium
      totam rem aperiam.</p>
  </body>
</html>
```

HTML